

Domain B: Retail Banking — Complaints and Conduct

Project overview

Build an internal complaints assistant for a fictional bank. It will help staff understand complaints, find relevant policies, compare previous cases and recommend next steps.

Build fresh inside the existing `project` folder in the course Git repository, following the structure of the Firm Intelligence API. The API and Streamlit UI will be separate projects inside that folder.

What the project must include

Part	Requirements from the brief	How this applies to Domain B
Records and API	Health endpoint; at least two entity types; create, read, update and delete; at least two query filters; Pydantic validation.	Planned entities: customers, complaints and products , covering all three listed under B. Filters could include product, severity and status.
Synthetic data	At least five records per entity and eight realistic prose documents . All data must be invented.	Include open and resolved complaints, customers with different needs, product terms, complaint policies, fictional guidance and case decisions, and redress methodology.
Claude features	Plain summary, streaming summary, structured analysis validated with Pydantic, and token estimates or counts. Keep rules in the system prompt and data in the user message.	Summarise a complaint and classify it into theme, severity and recommended next step .
Document retrieval	Voyage embeddings, persistent Chroma, manual indexing with upsert by ID, and search-only results with scores.	Search the bank's synthetic complaint-handling documents.
Grounded answers	Cite supporting documents. Refuse before calling Claude when no result clears the chosen relevance floor.	Decline unsupported advice, particularly about redress—how the bank puts things right, including compensation.

Part	Requirements from the brief	How this applies to Domain B
Agent	Knowledge search plus another tool; hard iteration limit; safe handling of unknown tools, missing arguments and failures; return tool-call count and total token usage.	The additional tool must find similar historic complaints and their recorded outcomes.
Streamlit UI	Separate project and environment. Include Ask, Search only, Streaming summary and one original feature.	Suggested original feature: a complaint triage screen where staff filter and review cases.
Engineering	Environment-based configuration; provider errors mapped to 504 / 429 / 502 ; mocked tests; requirements and README files.	Handle missing records, unavailable providers, empty indexes and incomplete agent runs clearly.

Important details

- The Ask screen must show the answer, tool calls and tokens, with a distinct message when the agent does not complete.
- Search only must list every returned result with its title and score, and distinguish “**index not built**” from an ordinary failure.
- Streaming summary must let the user select a record and watch the summary arrive.
- Map provider timeouts to **504**, rate limits to **429**, and other upstream failures to **502**.
- Configure API keys, model names and thresholds through environment variables. Do not put secrets in code or Git.
- The data is synthetic, but application features must call real, running services. Mock LLM and embedding calls in tests.
- Label fictional guidance and ombudsman-style case decisions clearly as synthetic project material.

How this project must differ from Firm Intelligence

The brief requires six differences:

1. **Your own data model:** customers, products and complaints, with domain-specific fields and filters.
2. **Your own extra tool:** similar historic complaint lookup.
3. **Your own analysis schema:** complaint theme, severity and recommended next step.
4. **Your own refusal rule:** defined and proved in a test.
5. **A risk-sensitive or time-sensitive element:** proposed choice—human review before acting on recommended outcomes.
6. **An original UI feature:** proposed choice—a complaint triage screen.

The human-review rule and triage screen are proposed design choices; the six categories themselves are requirements.

Process and build order

Stage	What we'll do	What must work before moving on
1. Agree the specification	Define the user, supported complaint types, fields, refusal rules, original screen and example questions. Turn the brief into a checklist.	We can explain exactly what the finished app will do.
2. Set up the project	Create <code>complaints-api</code> and <code>complaints-ui</code> inside <code>project</code> , each with its own environment. Add Git ignore rules and configuration placeholders.	The structure is ready and secrets are excluded from Git.
3. Design the data	Create linked customer, product and complaint records, then write the eight or more documents. Include historic cases and questions the system should refuse.	The data supports the intended demonstrations.
4. Build the basic API	Add health, CRUD, filters, validation and missing-record errors.	We can manage all three record types through FastAPI's <code>/docs</code> .
5. Add Claude	Implement summary, streaming summary, structured analysis, token reporting and provider error handling.	Each feature works independently, with mocked summary tests.
6. Add retrieval and refusal	Implement Voyage embeddings, persistent Chroma, indexing, search and cited answers. Try different relevance floors.	Supported questions get evidence; weak retrieval causes refusal without a Claude call, proved by a test.
7. Add the agent	Build historic complaint lookup first, then connect both tools to the agent. Add limits and safe errors.	The agent can use both tools and stops correctly at its limit.
8. Build Streamlit	Connect the four views to the running API.	Every button uses real backend functionality and displays failures clearly.
9. Finish and rehearse	Complete tests, READMEs, dependencies, design note and demo.	Someone else can run it, and you can explain it end to end.

Test and commit working stages as we go, pushing project changes to the existing GitHub repository. Stage only the intended project changes so unrelated course work is not included accidentally.

Project location and familiar structure

- `project/complaints-api/` — FastAPI backend, records, documents, Claude logic, retrieval, agent and tests.
- `project/complaints-ui/` — separate Streamlit application with its own environment and dependencies.
- `project/docs/` — specification and required design note.

Use the same separation of responsibilities as Firm Intelligence:

File or folder	Intended responsibility
<code>main.py</code>	Create FastAPI, register routers and expose the health endpoint.
<code>data.py</code>	Hold the synthetic structured records.
<code>documents.py</code>	Hold the prose documents used for retrieval.
<code>llm.py</code>	Claude summaries, streaming, structured analysis and grounded generation.
<code>knowledge.py</code>	Voyage embedding functions.
<code>knowledge_store.py</code>	Persistent Chroma indexing and search.
<code>agent.py</code>	Tool-use loop and safe tool execution.
<code>routers/</code>	HTTP endpoints, request validation and response handling.
Tests	Verify required behaviour using mocked provider calls.

This is a proposed organisation based on the inspected API, not a requirement to copy its unfinished code.

Hand-in checklist

- ☐ API project with exact setup and run instructions.
- ☐ UI project with exact setup and run instructions.
- ☐ A `requirements.txt` for each project.
- ☐ Required environment variables documented without including secrets.
- ☐ At least five synthetic records per entity.
- ☐ At least eight realistic synthetic documents.
- ☐ Mocked tests covering summaries, refusal and the agent loop, including its iteration limit.
- ☐ One-page design note.
- ☐ Live demonstration of one question using **both tools**.
- ☐ Live demonstration of one question being **correctly refused**.

Required one-page design note

Answer these questions using evidence from the build:

1. Why did you choose your relevance floor, and what happened when you tried other values?
2. Would you chunk the documents? Why or why not, given their length and structure?
3. What is the worst wrong answer the system could give, and what stops it?

What the assessors prioritise

- Features genuinely calling the running API and services.
- Correct handling of provider errors, empty indexes, unknown records and agent limits.
- Realistic domain data, prompts and refusal behaviour.
- Clear differences from the law-firm version.
- Your ability to explain every button press end to end, including what goes to Voyage, Chroma and Claude.
- Your ability to justify the relevance floor, iteration limit and model choice.

Working features, sensible domain behaviour, failure handling and understanding take priority over visual polish.

Optional stretch goals

Only consider these after completing the required features:

- Chunk long documents and retain source document IDs in metadata.
- Filter retrieval by metadata such as document type or date.
- Rerank retrieved results.
- Stream grounded answers as well as summaries.
- Build a small retrieval evaluation set and report results.
- Add authentication and API rate limiting.
- Add Docker and Compose to run both projects.

Immediate next step

Stage 1: write a short specification and acceptance checklist. Agree the remaining design choices before creating the data or application code.